Subproyecto de Integración

1. Descripción

El objetivo del proyecto es desarrollar una herramienta de minería de datos que permita cargar, procesar y analizar información sobre creadores de contenido multimedia alojados en distintas plataformas. Para ello, se desarrollará una aplicación con una arquitectura de tres microservicios, mostrada en la Figura 1. Los servicios *PeerTubeMiner* y *DailymotionMiner* tendrán como objetivo leer datos de PeerTube y Dailymotion, respectivamente, y enviarlos a *VideoMiner* para su almacenamiento usando un modelo de datos común. Por su parte, *VideoMiner* almacenará los datos y los ofrecerá al exterior a través de una API REST para que otras aplicaciones puedan consultarlos y analizarlos.

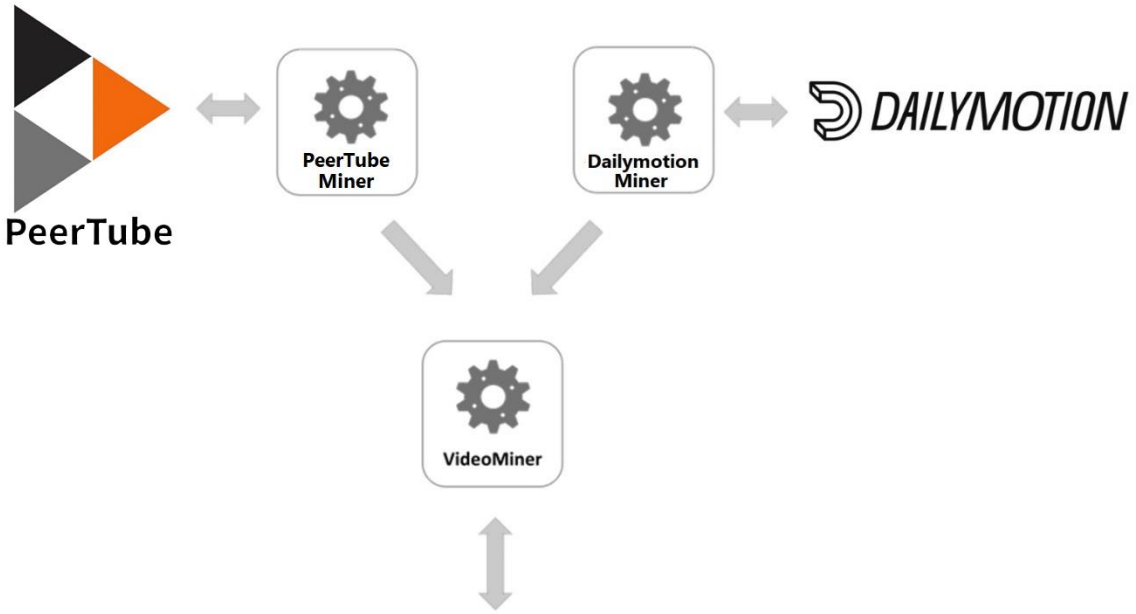

Figura 1. Vista general de la arquitectura

A continuación, se detalla el funcionamiento de cada servicio:

PeerTubeMiner. Se trata de un servicio adaptador que leerá los datos desde la API REST de PeerTube y los enviará a VideoMiner usando el modelo de datos adecuado (Figura 2). En un escenario de ingeniería de datos real este proceso podría realizarse periódicamente (ej. cada 24 horas). En este caso, el servicio implementará un servicio RESTful con una operación para obtener los datos de un canal de PeerTube y enviarlos a VideoMiner cada vez que sea invocada. En concreto, la operación deberá poder ser invocada de la siguiente manera:

POST *apipath*/{id}[?maxVideos=10&maxComments=2]

Parámetros opcionales:

- **maxVideos**: La operación devolverá el número de vídeos por canal introducido como parámetro. Valor por defecto: 10.

- **maxComments**: La operación devolverá el número de comentarios por video introducido como parámetro. Valor por defecto: 2.

Se recomienda implementar también una operación GET equivalente de sólo lectura para hacer pruebas, es decir, que muestre los resultados de la búsqueda sin enviarlos a VideoMiner.

DailymotionMiner. Su funcionamiento será análogo al del servicio anterior, pero en este caso la información se tomará de Dailymotion, a través de su API REST.

POST *apipath*/{id}[?maxVideos=10&maxPages=2]

Parámetros opcionales:

- **maxVideos**: La operación devolverá el número de videos por canal introducido como parámetro. Valor por defecto: 10.
- **maxPages**: Número máximo de páginas de resultados a devolver. Valor por defecto: 2.

VideoMiner. Será el servicio responsable de integrar y almacenar todos los datos en una base de datos H2 usando el modelo de datos mostrado en la Figura 2. Para ello, implementará una API REST que permitirá manipular los siguientes recursos:

- Channel. Implementará varias operaciones de lectura y escritura para añadir y listar canales. Entre otras, tendrá que implementar la operación POST adecuada para que los adaptadores puedan añadir datos de nuevos canales.
- Videos. Implementará, como mínimo, varias operaciones de lectura para listar todos los vídeos, y buscar los vídeos por id.
- Captions. Implementará, como mínimo, varias operaciones de lectura para listar todas las captions, buscarlas por id y devolver las captions de un vídeo dado su id.
- Comments. Implementará, como mínimo, varias operaciones de lectura para listar todos los comentarios, buscarlos por id y devolver los comentarios de un vídeo dado su id.

Como material de apoyo, se proporciona:

1. Proyecto plantilla con las clases del modelo de datos de VideoMiner, necesarias para la creación de la base de datos H2 y la manipulación de los datos recibidos desde los servicios adaptadores.
2. Conjunto de pruebas de Postman para validar el funcionamiento básico de VideoMiner.
3. Video demo ilustrando el comportamiento esperado de la aplicación y cómo probarla.

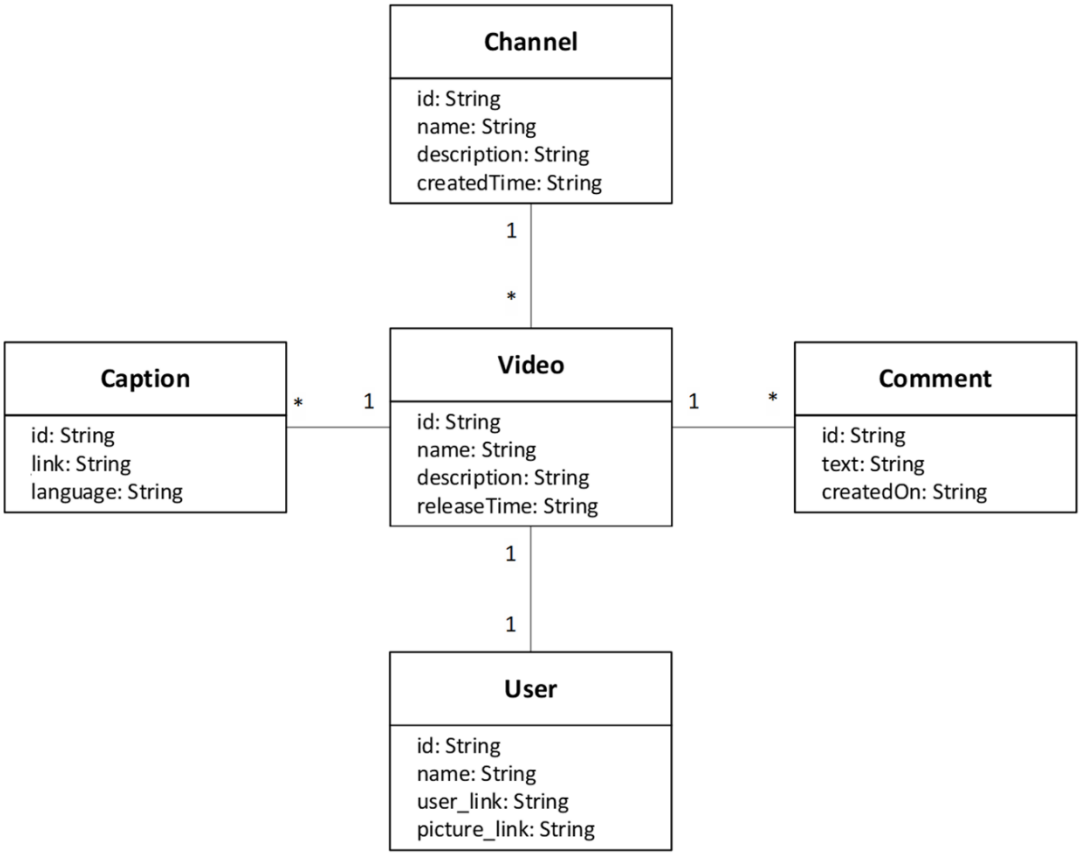

Figura 2. Modelo de datos

Consideraciones importantes:

- Las captions se denominan “Subtitle” en la API de Dailymotion.
- El usuario se denomina “Account” en PeerTube y “Owner” en Dailymotion.
- Los comentarios en PeerTube se denominan “Comment threads”. No existen comentarios en la API de Dailymotion, en su defecto usar los tags del video.
- La aplicación implementada debe seguir las buenas prácticas de REST. Por ejemplo, si pedimos a cualquiera de los tres servicios que nos devuelva un recurso con un id que no existe, el servicio debe devolver un código de respuesta 404.

2. Criterios de evaluación

2.1. Criterios mínimos (IMPRESCINDIBLES PARA APROBAR)

Los tres servicios deberán funcionar correctamente de acuerdo con las instrucciones indicadas. Es imprescindible que todas las pruebas de Postman facilitadas como parte del material de apoyo pasen con éxito. No se aceptarán aplicaciones que no se ajusten a la arquitectura descrita anteriormente.

2.2. Criterios estándar (necesarios para obtener una calificación de notable)

Además de los criterios mínimos, se valorarán los aspectos de diseño de la aplicación, así como el nivel de detalle de las pruebas (unitarias con Junit y de sistema con Postman), la documentación interactiva de la API (OAS). También se tendrá en cuenta el nivel de participación y la claridad de la exposición durante la defensa.

2.3. Criterios avanzados (necesarios para obtener una calificación de sobresaliente)

Además de los criterios estándar, aquellos grupos que quieran optar a sobresaliente deberán implementar unas o varias mejoras acordadas con su tutor/a. Algunos ejemplos:

- Extensión del modelo de datos.
- Extensión de la API proporcionada por VideoMiner (ej. paginación, filtros, ordenación).
- Extraer datos de más plataformas (ej. Twitch).
- Uso de APIs GraphQL.
- Uso de autenticación mediante API key en VideoMiner.
- Desplegar los tres servicios en la nube.